

BÁO CÁO THỰC HÀNH

Thực hành An toàn mạng máy tính

Lab 1 (Phần 2): Mật mã học Cổ điển

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 29/09/2026

Nhóm 2 – NT101.R12.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo
24521073	Nguyễn Quang Minh	Câu 6
24521403	Võ Thành Phúc	Câu 3, câu 7

Nhiệm vụ 2.3

Nhiệm vụ 2.3

Bạn hãy xây dựng một chương trình hỗ trợ giải mã tự động mono-alphabetic substitution cipher khi chỉ biết ciphertext. Chương trình cần cho phép nhập một ciphertext bất kỳ hoặc đọc từ file và thực hiện giải mã tự động để tìm ra bản rõ có khả năng đúng cao nhất dựa trên các đặc trưng thống kê của ngôn ngữ. Kết quả cần hiển thị bản rõ tốt nhất tìm được và thông tin cơ bản về quá trình thực hiện. Bạn cần kiểm thử chương trình với ciphertext ở Task 3.1 hoặc các văn bản khác và nhận xét về hiệu quả của phương pháp.

Hình 1.1 : Yêu cầu đề bài 2.3

Gợi ý

- Ciphertext nên được chuẩn hóa trước khi xử lý, ví dụ chuyển về cùng một kiểu chữ và loại bỏ các ký tự không cần thiết để việc tính toán thống kê chính xác hơn.
- Cần thiết kế chương trình theo hướng có thể lặp lại nhiều lần quá trình tìm kiếm để cải thiện kết quả nếu cần.
- Có thể sử dụng các thước đo thống kê như Index of Coincidence hoặc các mô hình tần suất unigram, bigram hay trigram để đánh giá mức độ “giống tiếng Anh” của một văn bản.
- Để tìm khóa phù hợp, có thể áp dụng các phương pháp tìm kiếm hoặc tối ưu như hill-climbing, random restart, simulated annealing hoặc các kỹ thuật tương tự.

Hình 1.2 Gợi ý

1. Cơ sở lý thuyết: Thống kê ngôn ngữ (N-gram) và Hàm mục tiêu (Fitness Score)

Cách giải quyết: Bài toán bẻ khóa mật mã thể đơn bảng khi chỉ có ciphertext được chuyển thành một **bài toán tối ưu hóa**. Cần tìm khóa K trong số $26! \approx 4,03 \times 10^{26}$ khóa có thể, sao cho khi dùng K để giải mã, văn bản thu được “giống tiếng Anh nhất”. Không gian khóa quá lớn để duyệt cạn, nên cần một thước đo định lượng độ “giống tiếng Anh” để hướng dẫn quá trình tìm kiếm.

Ý tưởng chính: Ngôn ngữ tự nhiên không được tạo từ các chữ cái ngẫu nhiên. Trong tiếng Anh, một số chuỗi 4 chữ cái (quadgram) xuất hiện rất thường xuyên

(TION, THAT, THER), trong khi nhiều chuỗi khác gần như không tồn tại (ZZZX). Một bản rõ đúng sẽ chứa nhiều quadgram phổ biến, còn một bản giải mã sai sẽ chứa nhiều quadgram vô nghĩa.

Cơ sở lý thuyết (Log-Likelihood): Xác suất để một văn bản $T = t_1 t_2 \dots t_n$ là tiếng Anh được xấp xỉ bằng tích xác suất của các quadgram tạo nên nó:

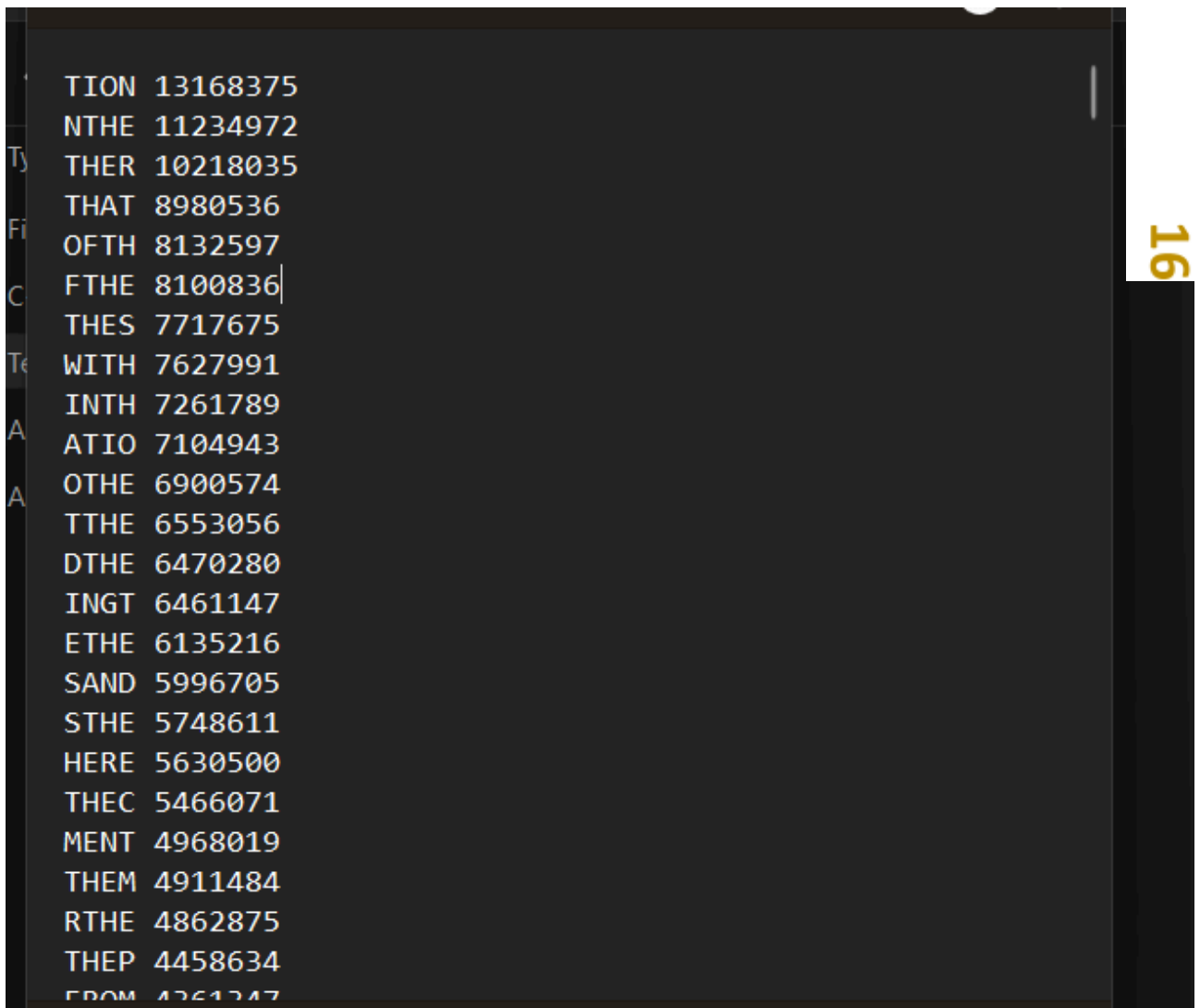
$$P(T) \approx \prod P(t_i t_{i+1} t_{i+2} t_{i+3}), \text{ với } i = 1 \dots n-3$$

Tích của rất nhiều số thập phân nhỏ sẽ gây tràn số dưới (underflow). Vì vậy ta dùng logarit để chuyển phép nhân thành phép cộng:

$$\text{FitnessScore}(T) = \sum \log_{10} P(t_i t_{i+1} t_{i+2} t_{i+3})$$

Điểm Fitness luôn âm. Điểm càng cao (càng gần 0) thì văn bản càng giống tiếng Anh.

Dữ liệu tần suất tĩnh được khai báo trong file `english_quadgrams.txt`, mỗi dòng gồm một quadgram và số lần xuất hiện của nó (ví dụ TION 13168375).



Hình 1.3: Một phần file english_quadgrams.txt

Xây dựng bảng điểm: Việc chuyển số đếm thành log-xác suất được thực hiện trong hàm loadQuadgrams(). Hàm làm ba việc:

- Đọc toàn bộ file và cộng dồn số đếm của từng quadgram vào mảng $26^4 = 456.976$ phần tử. Chỉ số của quadgram abcd là $((a \cdot 26 + b) \cdot 26 + c) \cdot 26 + d$.
- Tính tổng số đếm total.
- Gán điểm $\log_{10}(\text{count} / \text{total})$ cho mỗi quadgram có trong file. Quadgram chưa từng xuất hiện nhận mức phạt cố định $\log_{10}(0, 01 / \text{total})$, để tránh $\log(0) = -\infty$.

Bảng điểm lưu dưới dạng mảng phẳng QG[], nên tra cứu mất $O(1)$.

```

bool loadQuadgrams(const string& path) {
    ifstream f(path);
    if (!f) return false;
    vector<double> cnt(26 * 26 * 26 * 26, 0.0);
    double total = 0;
    string w;
    double c;
    while (f >> w >> c) {
        if (w.size() != 4) continue;
        int idx = 0;
        bool ok = true;
        for (char ch : w) {
            ch = toupper((unsigned char)ch);
            if (ch < 'A' || ch > 'Z') { ok = false; break; }
            idx = idx * 26 + (ch - 'A');
        }
        if (!ok) continue;
        cnt[idx] += c;
        total += c;
    }
    if (total <= 0) return false;
    float floorLp = (float)log10(0.01 / total); // quadgram chưa từng gặp
    QG.assign(cnt.size(), floorLp);
    for (size_t i = 0; i < cnt.size(); i++)
        if (cnt[i] > 0) QG[i] = (float)log10(cnt[i] / total);
    return true;
}

```

Hình 1.4: Hàm loadQuadgrams()

Chấm điểm văn bản: Việc chấm điểm nằm ở hàm scoreText(ct, key, buf). Hàm giải mã ciphertext bằng khóa hiện tại vào bộ đệm buf, sau đó trượt cửa sổ 4 ký tự qua toàn bộ văn bản và cộng dồn điểm log của từng quadgram. Độ phức tạp là $O(n)$ với n là độ dài văn bản.

```

double scoreText(const vector<int>& ct, const Key& key, vector<int>& buf) {
    size_t n = ct.size();
    for (size_t i = 0; i < n; i++) buf[i] = key[ct[i]];
    double s = 0;
    for (size_t i = 0; i + 3 < n; i++)
        s += QG[(buf[i] * 26 + buf[i + 1]) * 26 + buf[i + 2]] * 26 + buf[i + 3];
    return s;
}

```

Hình 1.5: Hàm scoreText()

2. Chuẩn hóa dữ liệu (Preprocessing)

Ý tưởng: Để đếm quadgram chính xác, dữ liệu phải đồng nhất. That, THAT và t.h, a, t đều phải được xem là THAT.

Cơ sở lý thuyết: Tiền xử lý loại bỏ nhiễu (khoảng trắng, dấu câu, chữ số) và đưa về cùng một kiểu chữ (chữ in hoa). Nhờ đó, các quadgram trong văn bản khớp hoàn hảo với bảng quadgram chuẩn. Nếu giữ lại dấu cách hoặc dấu câu, ta sẽ tạo ra các quadgram như S, I hay T WA không có trong bảng, làm sai lệch điểm số.

Thực hiện: Trong `main()`, chương trình duyệt từng ký tự của `ciphertext`, chỉ giữ ký tự A–Z và đổi sang chữ hoa. Mỗi chữ cái được đổi thành số 0–25 và lưu vào `vector<int> ct`, giúp các phép tra cứu sau đó chạy nhanh. Nếu `ciphertext` ngắn hơn 20 chữ cái, chương trình báo lỗi vì thống kê không đủ tin cậy. Văn bản gốc (kèm dấu câu, khoảng trắng, chữ hoa/thường) được giữ nguyên để hàm `applyKey()` hiển thị bản rõ đúng định dạng ban đầu.

```
// ---- Chuẩn hoá
vector<int> ct;
for (char ch : original)
    if (isalpha((unsigned char)ch) && toupper((unsigned char)ch) <= 'Z')
        ct.push_back(toupper((unsigned char)ch) - 'A');
if (ct.size() < 20) {
    cerr << "Ciphertext quá ngắn (" << ct.size() << " chữ cái) - cần tối thiểu ~100 để có kết quả tin cậy.\n";
    return 1;
}

// Thống kê ciphertext
```

Hình 1.6: Đoạn code chuẩn hóa trong `main()`

3. Giải thuật tìm kiếm cục bộ: Leo đồi (Hill Climbing)

Ý tưởng: Không gian khóa 26! quá lớn để duyệt cạn. Ta bắt đầu từ một khóa ban đầu, sau đó thử thay đổi một chút. Nếu sự thay đổi làm văn bản có nghĩa hơn (điểm Fitness tăng), ta giữ lại thay đổi đó.

Biểu diễn khóa: `Key[c]` là chữ cái bản rõ ứng với chữ cái mã `c`. Khóa là một hoán vị của 26 chữ cái. "Khóa lân cận" được tạo bằng cách **hoán đổi hai phần tử** của khóa, tức là đổi phép ánh xạ của hai chữ cái mã.

Khóa khởi tạo: Hàm `freqInitKey()` đếm tần suất từng chữ cái trong `ciphertext` rồi gán theo thứ tự tần suất tiếng Anh E T A O I N S H R D L C U M W F G Y P B V K J X Q Z. Chữ mã xuất hiện nhiều nhất được ánh

xạ thành E, nhiều thứ nhì thành T, và tiếp tục như vậy. Cách này cho điểm xuất phát đã khá gần đáp án.

Cơ sở lý thuyết: Thuật toán trong hàm `hillClimb()` hoạt động như sau:

1. Lấy khóa hiện tại K và điểm hiện tại cur.
2. Lần lượt thử tất cả 325 cặp hoán đổi (i, j) với $0 \leq i < j < 26$, mỗi lần tạo một khóa lân cận K'.
3. Nếu $\text{Fitness}(K') > \text{Fitness}(K)$ thì cập nhật $K = K'$ (leo lên một bước). Ngược lại thì hoán đổi lại.
4. Lặp lại các lượt duyệt cho đến khi một lượt đầy đủ không cải thiện được nữa. Lúc này K là **cực đại địa phương**.

```
double hillClimb(Key& key, const vector<int>& ct, vector<int>& buf, double cur) {
    bool improved = true;
    while (improved) {
        improved = false;
        for (int i = 0; i < 25; i++)
            for (int j = i + 1; j < 26; j++) {
                swap(key[i], key[j]);
                double s = scoreText(ct, key, buf);
                if (s > cur + 1e-9) { cur = s; improved = true; }
                else swap(key[i], key[j]);
            }
    }
    return cur;
}
```

Hình 1.7: Hàm `hillClimb()`

4. Khắc phục cực đại địa phương: Random Restart và Shake

Vấn đề: Hill Climbing chỉ nhận thay đổi làm tăng điểm nên dễ mắc kẹt ở cực đại địa phương, là khóa gần đúng nhưng chưa phải khóa đúng.

Giải pháp: Chương trình được thiết kế để lặp lại quá trình tìm kiếm nhiều lần:

- **Random restart:** Mỗi lần restart khởi tạo một khóa mới. Lần đầu dùng khóa tần suất. Các lần sau xen kẽ giữa khóa hoàn toàn ngẫu nhiên và khóa tần suất bị nhiễu 10 hoán đổi. Kết quả của mỗi lần restart được so sánh với khóa tốt nhất toàn cục.
- **Shake (iterated local search):** Từ cực đại địa phương, chương trình thực hiện 2–3 hoán đổi ngẫu nhiên rồi chạy lại Hill Climbing. Nếu điểm

cao hơn thì giữ, ngược lại thì bỏ. Nhờ vậy thuật toán thoát khỏi vùng lân cận hẹp mà không mất thành quả đã đạt được.

Số restart (-r) và số lần shake (-k) có thể điều chỉnh để đánh đổi giữa thời gian chạy và độ tin cậy.

5. Các thước đo hỗ trợ: Index of Coincidence

Trước khi tìm khóa, chương trình tính **Index of Coincidence (IC)** của ciphertext:

$IC = \sum n_i(n_i - 1) / (N(N - 1))$, với n_i là số lần xuất hiện của chữ cái thứ i và N là độ dài văn bản.

IC của tiếng Anh xấp xỉ 0,0667, còn văn bản chữ cái ngẫu nhiên xấp xỉ 0,0385. Mã thay thế đơn bằng **không đổi phân bố tần suất**, chỉ đổi tên chữ cái, nên IC của ciphertext vẫn gần 0,067. IC vì thế xác nhận ciphertext có thể là mã thay thế đơn bằng. Nếu IC gần 0,038, ta cần nghi ngờ loại mã khác (ví dụ Vigenère) trước khi áp dụng phương pháp này.

6. Quy trình tổng thể của chương trình

1. Đọc bảng quadgram (loadQuadgrams).
2. Đọc ciphertext, chuẩn hóa, in thống kê (độ dài, IC, các chữ cái phổ biến nhất).
3. Với mỗi lần restart: tạo khóa khởi tạo, chạy Hill Climbing, rồi lặp Shake + Hill Climbing.
4. Cập nhật khóa tốt nhất toàn cục, in ra mỗi khi tìm được điểm cao hơn.
5. In khóa, điểm quadgram, số lượt Hill Climbing, thời gian chạy và bản rõ tốt nhất (applyKey).

```
PS D:\ro\bai2cham3> ./mono_solver
-ÉiBñâm quadgram (log10) : -1577.46 (trung b|%nh -4.09 / quadgram; tiB||ng Anh th||BñYng ĩêê -4.2 -æB||γn -4.8)
SBñæ l||Bñút hill-climb : 9030
ThBñYi gian : 5.93 s

=== BB||GN R|ò TÈÉT NH8||AT ===
IT WAS THE BEST OF TIMES, IT WAS THE WORST OF TIMES, IT WAS THE AGE OF WISDOM, IT WAS THE AGE OF FOOLISHNESS, IT WAS THE EPOCH OF BELIEF, IT WAS TH
POCH OF INCRECULITY, IT WAS THE SEASON OF LIGHT, IT WAS THE SEASON OF DARKNESS, IT WAS THE SPRING OF HOPE, IT WAS THE WINTER OF DESPAIR, WE HAD EVE
HING BEFORE US, WE HAD NOTHING BEFORE US, WE WERE ALL GOING DIRECT TO HEAVEN, WE WERE ALL GOING DIRECT THE OTHER WAY. THERE WERE A KING WITH A LARG
AW AND A QUEEN WITH A PLAIN FACE, ON THE THRONE OF ENGLAND.
```

Hình 1.8: Giao diện kết quả khi chạy chương trình

Nhiệm vụ 2.6**Nhiệm vụ 2.6**

Bạn hãy xây dựng một chương trình hỗ trợ phá mã Vigenère cipher khi không biết khóa. Chương trình cần cho phép nhập hoặc đọc từ file một ciphertext bất kỳ và thực hiện quá trình phân tích để tự động tìm khóa và bản rõ có khả năng đúng cao nhất. Kết quả cần hiển thị khóa tìm được, bản rõ tương ứng và một số thông tin cơ bản về quá trình phân tích. Sinh viên sử dụng chương trình để giải mã ciphertext được cung cấp và trình bày kết quả. (<https://pastecode.io/s/wm8dsq5z>)

```
pp oiuibvql avpgzwm, vyabnwzycbbg klhqla mv uqwckl kzzwktcfwg hpp
wwftwzcktakah bxjjzcynlu pyzbcgp zzht omnpxtcfkts eahkxwve uvw
h uqn wy ywxy-jtzgp wiejwxubbvpe wiesgp utzvtunpfz va nztuurizf
tgemizlu uh etfu fbim htq bikk va xmvprtyz. mogey lxagdgqgpufck
tsialqmooe uzx buqx nhy edsxmviduxape wyg zlpqlimpqz uvw kkscbts
uuavbui mhl oltuzqvhvui mv rdibxjv pubt wtupivf, yqv jkvyecvz vp
fbm buvqlvxa czx khuhuxmgakmf khtoghqvhvui vl zwob il jtqxqm jedx
bkhpeukmpqzm igk gyuqe
```

Gợi ý

- Ciphertext nên được chuẩn hóa trước khi xử lý để việc phân tích chính xác hơn. Chương trình cần được thiết kế sao cho có thể thử nhiều khả năng khác nhau đối với độ dài khóa nhằm cải thiện kết quả.
- Có thể ước lượng độ dài khóa bằng các phương pháp thống kê như Index of Coincidence hoặc Kasiski examination, sau đó tách ciphertext thành các nhóm ký tự theo độ dài khóa và thực hiện phân tích tần suất để suy ra từng ký tự của khóa.

Mật mã Vigenère là một phương pháp mã hóa văn bản theo thuật toán thay thế đa bảng ký tự (polyalphabetic substitution cipher). Khác với mật mã Caesar (chỉ dịch chuyển tất cả ký tự trong văn bản theo một khoảng cố định k), Vigenère sử dụng một từ khóa (key) lặp đi lặp lại để dịch chuyển từng ký tự trong bản rõ theo các khoảng dịch khác nhau.

Ra đời vào thế kỷ 16 và trong hàng trăm năm, nó được gọi là le chiffre indéchiffrable (mật mã không thể phá giải) cho đến khi các phương pháp phân tích thống kê ra đời vào thế kỷ 19.

Nguyên lý hoạt động (Mã hóa và Giải mã)

Giả sử làm việc trên bảng chữ cái tiếng Anh (26 ký tự, từ a=0 đến z=25):

1. Quy tắc Mã hóa

Mỗi ký tự bản rõ P_i được cộng với ký tự tương ứng của khóa K_i theo phép toán dư (modulo 26):

$$C_i = (P_i + K_i) \bmod 26.$$

2. Quy tắc Giải mã

Mỗi ký tự bản mã C_i được trừ đi ký tự tương ứng của khóa K_i :

$$P_i = (C_i - K_i + 26) \bmod 26$$

Cách xử lý bài này:

1. Chuẩn hóa data đầu vào

```
def clean_text(text):
    return ''.join(c.lower() for c in text if c.isalpha())
```

Cách thức làm việc: Hàm duyệt qua từng ký tự của chuỗi bản mã (ciphertext), chỉ giữ lại các ký tự chữ cái (loại bỏ dấu cách, dấu chấm, dấu phẩy, số...) và chuyển tất cả về chữ thường.

=> Giúp việc tính toán tần suất thống kê ở các bước sau đạt độ chính xác cao nhất mà không bị lẫn bởi các khoảng trắng hay ký tự đặc biệt

2. Ước lượng độ dài khóa

```
def calculate_ioc(text):
    n = len(text)
    if n <= 1: return 0
    freqs = Counter(text)
    ioc = sum(f * (f - 1) for f in freqs.values()) / (n * (n - 1))
    return ioc

def estimate_key_length(text, max_len=20):
    best_len = 1
    max_ioc = 0
    for k in range(1, max_len + 1):
```

```

avg_ioc = 0
for i in range(k):
    column = text[i::k]
    avg_ioc += calculate_ioc(column)
avg_ioc /= k
if avg_ioc > max_ioc:
    max_ioc = avg_ioc
    best_len = k
return best_len, max_ioc

```

Cách thức làm việc:

1. Hàm `estimate_key_length` lần lượt thử giả định độ dài khóa k chạy từ 1 đến 20.
2. Với mỗi k , nó chia bản mã thành k cột (chuỗi con gồm các ký tự cách nhau k vị trí, sử dụng cú pháp `text[i::k]`).
3. Hàm `calculate_ioc` tính **Chỉ số IoC** cho từng cột theo công thức:

$$IoC = \frac{\sum f_i(f_i - 1)}{n(n-1)}$$

4. Chương trình tính IoC trung bình cho cả k cột. Độ dài khóa k nào cho **IoC** trung bình cao nhất (gần mức 0.065 của tiếng Anh tiêu chuẩn) sẽ được chọn làm độ dài khóa dự đoán.

3. Tìm từng ký tự của khóa:

```

def find_key(text, key_length):
    key = ""
    for i in range(key_length):
        column = text[i::key_length]
        best_shift = 0
        min_chi_sq = float("inf")

        for shift in range(26):
            chi_sq = 0
            shifted_col = [chr(((ord(c) - 97 - shift) % 26) + 97) for c in column]
            counts = Counter(shifted_col)
            n = len(shifted_col)

            for char, expected_prob in ENGLISH_FREQ.items():
                expected_count = expected_prob * n

```

```

        actual_count = counts.get(char, 0)
        if expected_count > 0:
            chi_sq += ((actual_count - expected_count) ** 2) /
expected_count

        if chi_sq < min_chi_sq:
            min_chi_sq = chi_sq
            best_shift = shift

    key += chr(best_shift + 97)
    return key

```

Cách thức làm việc:

1. Tách bản mã thành k cột (mỗi cột ứng với một ký tự trong khóa).
2. Với từng cột, thử lần lượt 26 độ dịch chuyển (shift từ 0 đến 25).
3. Ứng với mỗi shift, giải mã cột đó và dùng **Kiểm định Chi-bình phương** (χ^2) để so sánh tần suất xuất hiện chữ cái thu được với bảng tần suất chữ cái tiếng Anh tiêu chuẩn:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

(Trong đó O_i là số lần xuất hiện thực tế, E_i là số lần xuất hiện kỳ vọng).

4. Giá trị shift nào tạo ra độ lệch χ^2 nhỏ nhất sẽ ứng với ký tự khóa đúng của cột đó.

4. Khôi phục bản rõ:

```

def decrypt_vigenere(ciphertext, key):
    plaintext = []
    key_idx = 0
    for char in ciphertext:
        if char.isalpha():
            is_upper = char.isupper()
            c_val = ord(char.lower()) - 97
            k_val = ord(key[key_idx % len(key)]) - 97
            p_val = (c_val - k_val) % 26
            p_char = chr(p_val + 97)
            plaintext.append(p_char.upper() if is_upper else p_char)
            key_idx += 1

```

```

else:
    plaintext.append(char)
return ".join(plaintext)

```

Cách thức làm việc:

1. Duyệt qua từng ký tự của bản mã gốc (vẫn giữ nguyên khoảng trắng và dấu câu).
2. Nếu ký tự là chữ cái, nó lấy giá trị số của ký tự bản mã C_i trừ đi giá trị số tương ứng của ký tự khóa K_i theo công thức:

$$P_i = (C_i - K_i + 26) \pmod{26}$$
3. Giữ nguyên định dạng in hoa/in thường ban đầu của ký tự và ghép lại thành văn bản hoàn chỉnh.

=>Hàm `break_vigenere_cipher(ciphertext)`

Gọi `clean_text()` để làm sạch bản mã.

Gọi `estimate_key_length()` để lấy độ dài khóa k .

Gọi `find_key()` với độ dài k tìm được để suy ra chuỗi khóa (key).

Gọi `decrypt_vigenere()` để giải mã và trả về kết quả (key, plaintext)

Kết quả

```

(.venv) minh@minh-ASUS-V16-V3607VH-V3607VH:~/NT101_MangMayTinh/thuc-hanh/lab1$ python3 Bai2.6_lab1.py
[-] Cleaning ciphertext...
[-] Estimating key length (1 to 20)...
[+] Key length: 12 (Avg IoC = 0.0698)
[-] Analyzing frequency to recover key...
[+] Predicted key: HCMUITHCMUIT
[-] Decrypting...

=== RESULTS ===
KEY       : hcmuithcmuit
PLAINTEXT :
in computer science, cryptography refers to secure information and communication techniques derived from mathematical concep
ts and a set of rule-based calculations called algorithms to transform messages in ways that are hard to decipher. these det
erministic algorithms are used for cryptographic key generation and digital signing and verification to protect data privacy
, web browsing on the internet and confidential communications such as credit card transactions and email
(.venv) minh@minh-ASUS-V16-V3607VH-V3607VH:~/NT101_MangMayTinh/thuc-hanh/lab1$

```

Gọi `clean_text()` để làm sạch bản mã.

Gọi `estimate_key_length()` để lấy độ dài khóa `k`.

Gọi `find_key()` với độ dài `k` tìm được để suy ra chuỗi khóa (`key`).

Gọi `decrypt_vigenere()` để giải mã và trả về kết quả (`key`, `plaintext`)
Gọi `clean_text()` để làm sạch bản mã.

Gọi `estimate_key_length()` để lấy độ dài khóa `k`.

Gọi `find_key()` với độ dài `k` tìm được để suy ra chuỗi khóa (`key`).

Gọi `decrypt_vigenere()` để giải mã và trả về kết quả (`key`, `plaintext`)
Gọi `clean_text()` để làm sạch bản mã.

Gọi `estimate_key_length()` để lấy độ dài khóa `k`.

Gọi `find_key()` với độ dài `k` tìm được để suy ra chuỗi khóa (`key`).

Gọi `decrypt_vigenere()` để giải mã và trả về kết quả (`key`, `plaintext`)
Gọi `clean_text()` để làm sạch bản mã.

Gọi `estimate_key_length()` để lấy độ dài khóa `k`.

Gọi `find_key()` với độ dài `k` tìm được để suy ra chuỗi khóa (`key`).

Gọi `decrypt_vigenere()` để giải mã và trả về kết quả (key, plaintext) Gọi `clean_text()` để làm sạch bản mã.

Gọi `estimate_key_length()` để lấy độ dài khóa k.

Gọi `find_key()` với độ dài k tìm được để suy ra chuỗi khóa (key).

Gọi `decrypt_vigenere()` để giải mã và trả về kết quả (key, plaintext) Gọi `clean_text()` để làm sạch bản mã.

Gọi `estimate_key_length()` để lấy độ dài khóa k.

Gọi `find_key()` với độ dài k tìm được để suy ra chuỗi khóa (key).

Gọi `decrypt_vigenere()` để giải mã và trả về kết quả (key, plaintext)

Nhiệm vụ 2.7 :

Nhiệm vụ 2.7

Sinh viên lựa chọn một thuật toán mật mã cổ điển bất kỳ, trình bày ngắn gọn nguyên lý hoạt động và xây dựng một chương trình để thực hiện mã hóa và giải mã theo thuật toán đã chọn. Cần chuẩn bị một ví dụ minh họa để chứng minh chương trình hoạt động đúng, có thể bằng cách mã hóa và giải mã một đoạn văn bản mẫu.

Hình 3.1 : Đề bài 2.7

Rail Fence Cipher (mật mã hàng rào)

1. Nguyên lý hoạt động

Rail Fence là mật mã **hoán vị** (transposition): các ký tự của bản rõ không bị thay đổi, chỉ bị sắp xếp lại vị trí. Khóa là **số hàng** k (số "thanh ray").

Mã hóa:

- Viết bản rõ theo đường **zigzag** trên k hàng: đi xuống từ hàng 1 đến hàng k , rồi đi lên ngược lại, cứ thế lặp lại.
- Đọc lần lượt từng hàng, từ hàng 1 đến hàng k , ta được bản mã.

Giải mã:

- Biết độ dài bản mã n và khóa k , dựng lại đường zigzag để biết mỗi vị trí trong bản rõ thuộc hàng nào.
- Chia bản mã thành các đoạn theo số ký tự của từng hàng, rồi điền các ký tự trở lại đúng vị trí zigzag và đọc theo thứ tự ban đầu.

Cài đặt trong chương trình: hàm `_rail_pattern` tạo dãy chỉ số hàng (ví dụ $k = 3$: $0, 1, 2, 1, 0, 1, 2, 1, \dots$). Mã hóa gom ký tự theo hàng. Giải mã sắp xếp các vị trí theo (hàng, vị trí) rồi gán lần lượt ký tự bản mã vào các vị trí đó.

Đặc điểm: không gian khóa rất nhỏ (chỉ k từ 2 đến $n-1$) nên dễ bị tấn công vét cạn. Tần suất chữ cái không đổi so với bản rõ nên không an toàn, chỉ có giá trị học tập.

Các hàm chính:

- `railPattern(length, rails)` tạo dãy chỉ số hàng theo đường zigzag ($k = 3$ cho $0, 1, 2, 1, 0, 1, 2, \dots$).
- `encrypt(plain, rails)` đưa từng ký tự vào hàng tương ứng rồi nối các hàng lại.
- `decrypt(cipher, rails)` dựng lại zigzag, dùng `stable_sort` xếp các vị trí theo hàng (tức thứ tự đọc của bản mã), rồi gán lần lượt ký tự bản mã về đúng vị trí gốc.
- `showFence(text, rails)` in sơ đồ zigzag để minh họa.

2. Chạy chương trình

2.1 Mã hóa và Giải mã với Khóa $k = 3$

- Bản rõ:** TRUONG DAI HOC UIT KHOA MANG MAY TINH VA TRUYENTHONG
- Số hàng (k):** 3

Sơ đồ Zigzag trực quan:

```

So hang : 3
So do zigzag:
T . . . N . . . A . . . O . . . I . . . H . . . M . . . _ . . . _ . . . H . .
. . . Y . . . H . . .
. R . O . G . D . I . H . C . U . T . K . O . _ . A . G . M . Y . T . N . _ .
T . U . E . T . O . G
. . U . . . _ . . . _ . . . _ . . . _ . . . A . . . N . . . A . . . I . . . V
. R . . . N . . . N .

```

Hình 3.2 Kết quả thuật toán sơ đồ zigzag

Bản mã thu được:

TNAOT OAMT VUTORGD IHC IKAMN MYI HAR YNHNU I GAN TEO

Bản giải mã:

TRUONG DAI HOC UIT KHOA MANG MAY TINH VA TRUYENTHONG

Kết quả đối sánh: Thành công (d == msg \$to\$ true).

2.2 Thử nghiệm với các khóa $k = 2, 3, 4, 5$

Chương trình tiến hành thử nghiệm cùng một văn bản mẫu trên nhiều khóa khác nhau:

Khóa (k)	Bản mã (Ciphertext)	Kết quả giải mã	Trạng thái
$k = 2$	TUO DA HO UI KOAAN A IN A RUETONGROG I C T HMGMYTH V TYYNHO	TRUONG DAI HOC UIT KHOA MANG MAY TINH VA TRUYENTHONG	OK
$k = 3$	TNAOT OAMT VUTORGD IHC IKAMN MYI HAR YNHNU I GAN TEO	TRUONG DAI HOC UIT KHOA MANG MAY	OK

		TINH VA TRUYENTHONG	
$k = 4$	T D U A A A V TONGACIKAGMITHRNYNROIH O O MNYN UEO R H	TRUONG DAI HOC UIT KHOA MANG MAY TINH VA TRUYENTHONG	OK
$k = 5$	T A O T R H I A N Y N U ROGICIMMNIARYHOR D OKA MT V TEO UN G	TRUONG DAI HOC UIT KHOA MANG MAY TINH VA TRUYENTHONG	OK

```

-> TRUONG DAI HOC UIT KHOA MANG MAY TINH VA TRUYENTHONG [OK]
k=3: TNAOIHM H YHROGDIHCUTKO AGMYTN ATUETOGU ANAIVRNN
-> TRUONG DAI HOC UIT KHOA MANG MAY TINH VA TRUYENTHONG [OK]
k=4: T O MAHRHRGDHCTK AMYN TUTOUNA IHAN IV YNNOIUOGTAEG
-> TRUONG DAI HOC UIT KHOA MANG MAY TINH VA TRUYENTHONG [OK]
k=5: TAIM HRDIUT AYTATTOU ANAIVRNNOGHCKOGMN UEGNOH HY
-> TRUONG DAI HOC UIT KHOA MANG MAY TINH VA TRUYENTHONG [OK]

Kiem thu gau nhien 1000 truong hop: DAT

```

Hình ảnh 3.3 : Kết quả của thử nghiệm 2

HẾT